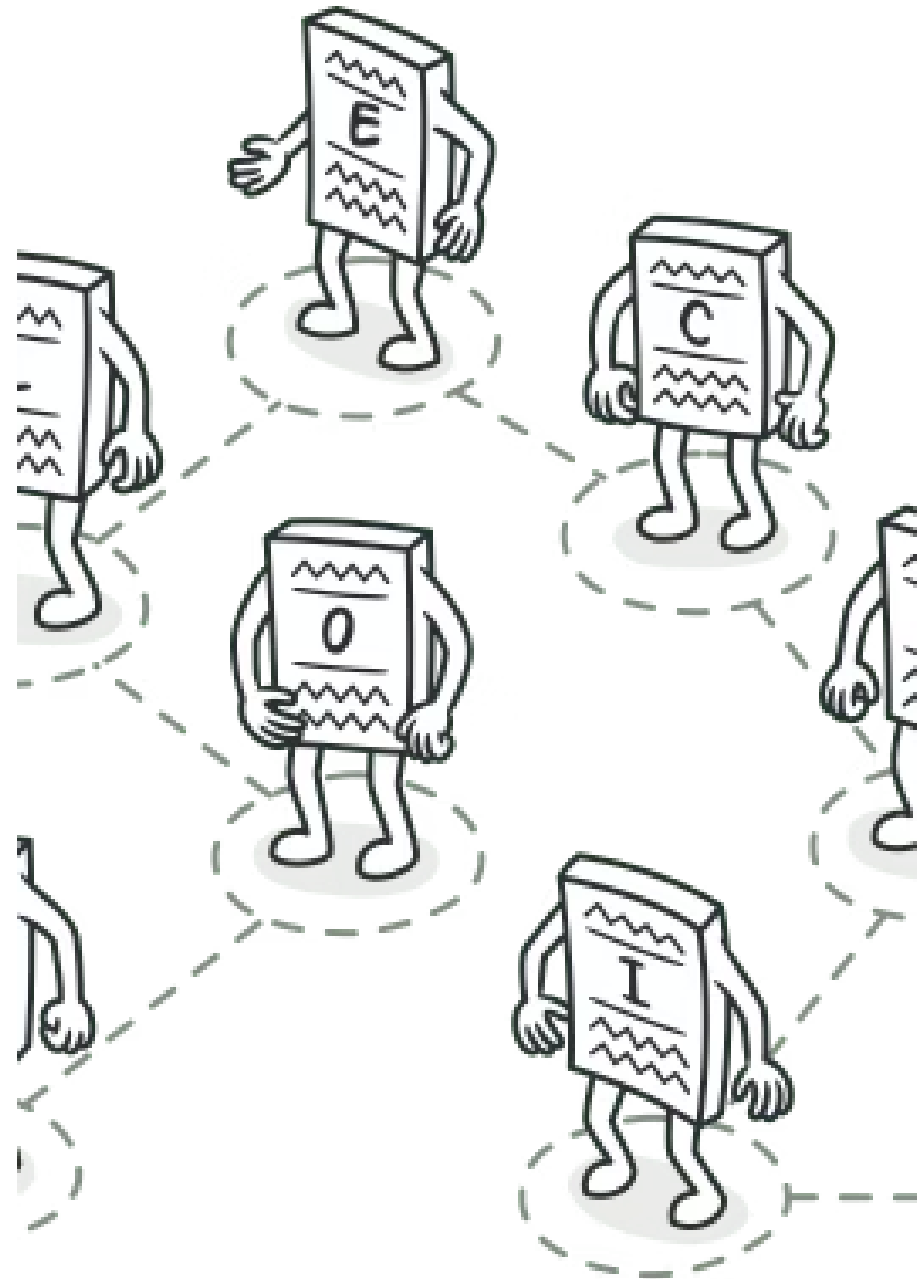


PATRÓN DE DISEÑO DE COMPORTAMIENTO

Iterator

También llamado: **Iterador**. Un patrón que te permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).



Recorrido por el Patrón Iterator

Exploraremos en detalle cómo este patrón simplifica el acceso a datos y desacopla la iteración de la estructura de la colección.

01

Introducción

Propósito, el problema de las colecciones y la solución del iterador.

02

Conceptos Clave

Interfaz común, analogías y formas de iterar.

03

Estructura Detallada

Componentes del patrón y su interacción.

04

Ejemplo Práctico

Implementación con perfiles sociales y pseudocódigo.

05

Aplicación y Conclusiones

Cuándo usarlo, ventajas, desventajas y relación con otros patrones.

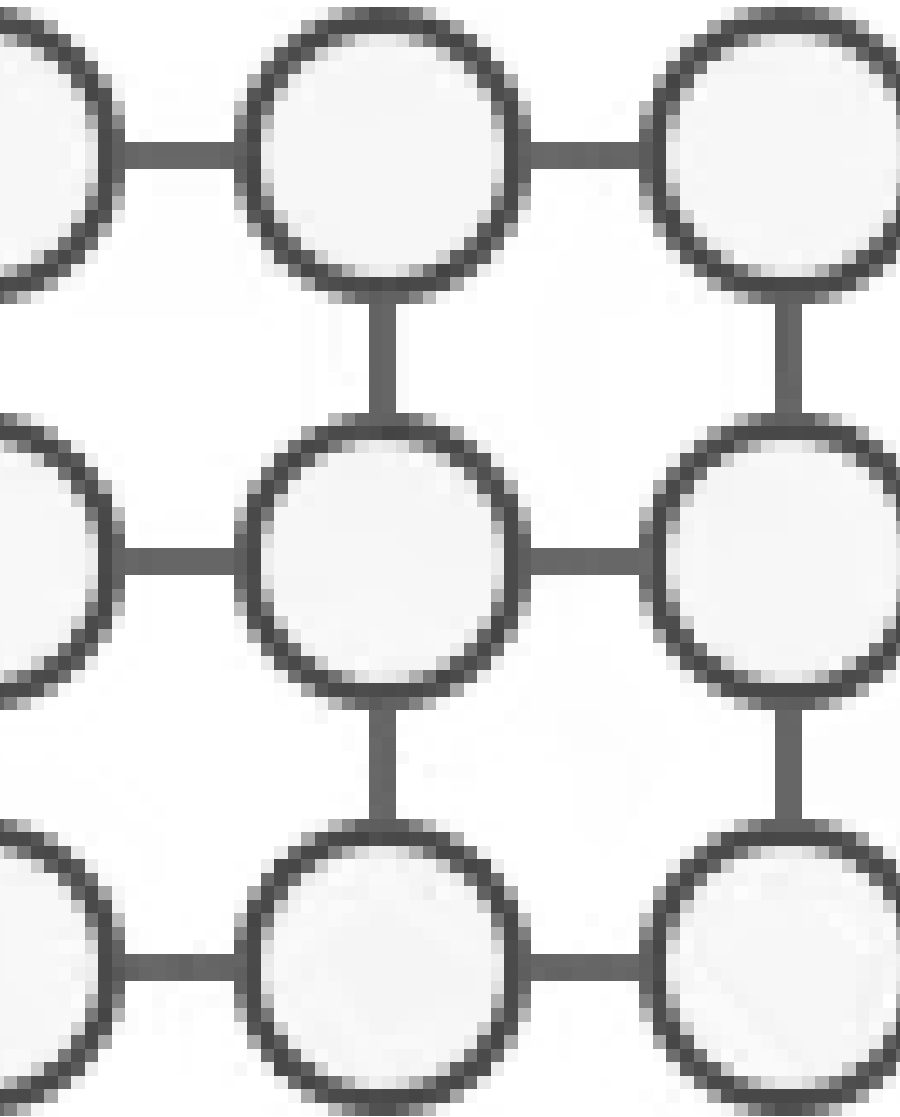
An abstract geometric pattern on the left side of the page, featuring a network of thin, light blue lines connecting various nodes. The nodes are represented by small circles and polygons in shades of blue, green, and white. Some lines are thicker and more prominent, creating a sense of depth and structure. The overall aesthetic is clean and modern, typical of a technical or design-oriented book cover.

CAPÍTULO 1

Propósito del Patrón Iterator

Iterator es un **patrón de diseño de comportamiento** que te permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).

Su objetivo principal es desacoplar el algoritmo de recorrido de la estructura interna de la colección, proporcionando una interfaz uniforme para navegar por cualquier tipo de estructura de datos.



CAPÍTULO 2 · EL PROBLEMA

El Problema: Las Colecciones

Las colecciones son de los tipos de datos más utilizados en programación. Sin embargo, una colección tan solo es un **contenedor para un grupo de objetos**.

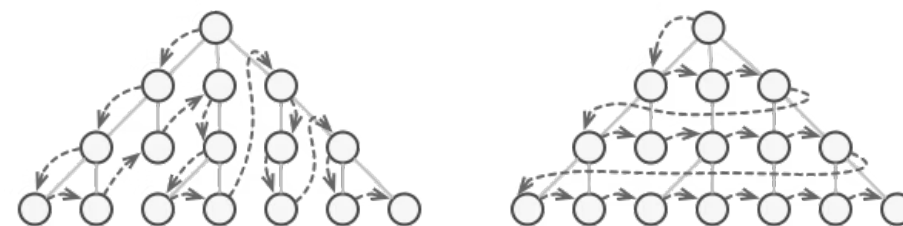
La mayoría almacena sus elementos en simples listas, pero algunas se basan en **pilas, árboles, grafos** y otras estructuras complejas de datos.

¿Cómo Acceder a los Elementos?

Independientemente de cómo se estructure una colección, debe aportar una forma de acceder a sus elementos de modo que otro código pueda utilizarlos, **sin acceder a los mismos elementos una y otra vez**.

Si tienes una colección basada en una lista, solo recorres en bucle todos sus elementos. Pero, ¿cómo recorres secuencialmente elementos de una estructura compleja como un **árbol**?

- Un día puede bastarte con un **recorrido de profundidad**
- Al día siguiente, quizá necesites un **recorrido en anchura**
- La semana siguiente, un **acceso aleatorio** a los elementos



La misma colección puede recorrerse de varias formas diferentes.

Problemas al Mezclar Responsabilidades

Responsabilidad nublada

Añadir más algoritmos de recorrido a la colección nubla gradualmente su responsabilidad principal: el **almacenamiento eficiente** de la información.

Algoritmos específicos

Algunos algoritmos pueden estar personalizados para una aplicación específica, por lo que incluirlos en una clase genérica de colección resulta **extraño**.

Acoplamiento del cliente

El código cliente que funciona con varias colecciones puede no saber cómo éstas almacenan sus elementos, forzando el **acoplamiento** a clases específicas.

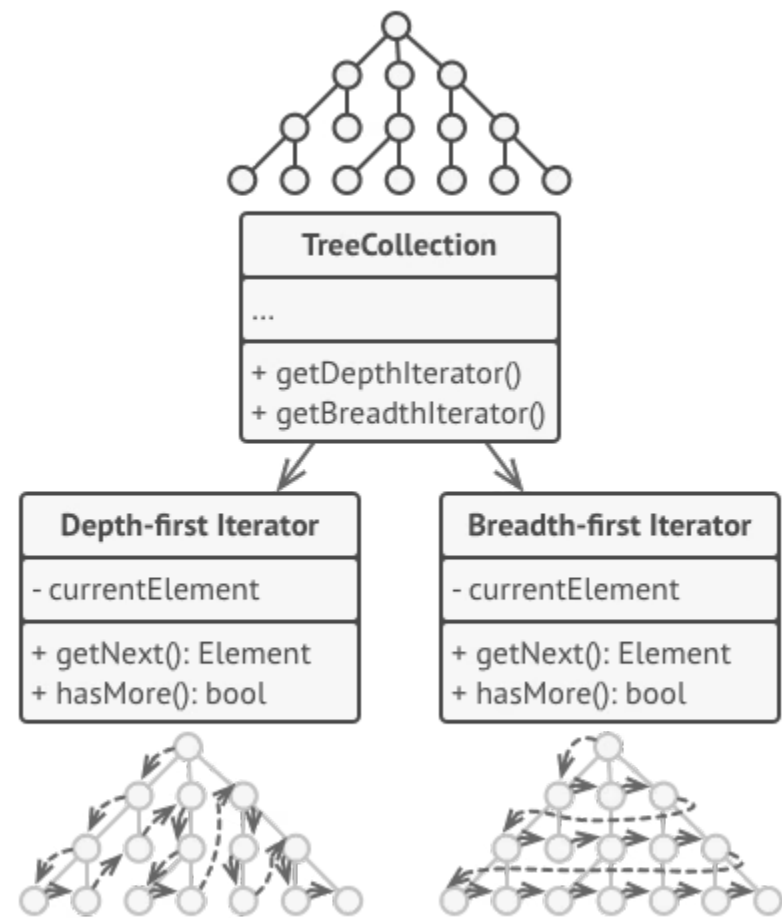
La Solución: El Objeto Iterador

La idea central del patrón Iterator es **extraer el comportamiento de recorrido** de una colección y colocarlo en un objeto independiente llamado **iterador**.

Además de implementar el propio algoritmo, un objeto iterador encapsula todos los detalles del recorrido:

- La **posición actual**
- Cuántos **elementos quedan** hasta el final

Varios iteradores pueden recorrer la **misma colección al mismo tiempo**, independientemente los unos de los otros.



Los iteradores implementan varios algoritmos de recorrido. Varios objetos iteradores pueden recorrer la misma colección al mismo tiempo.

Interfaz Común para Iteradores

Normalmente, los iteradores aportan un **método principal** para extraer elementos de la colección. El cliente puede continuar ejecutando este método hasta que no devuelva nada, lo que significa que el iterador ha recorrido todos los elementos.

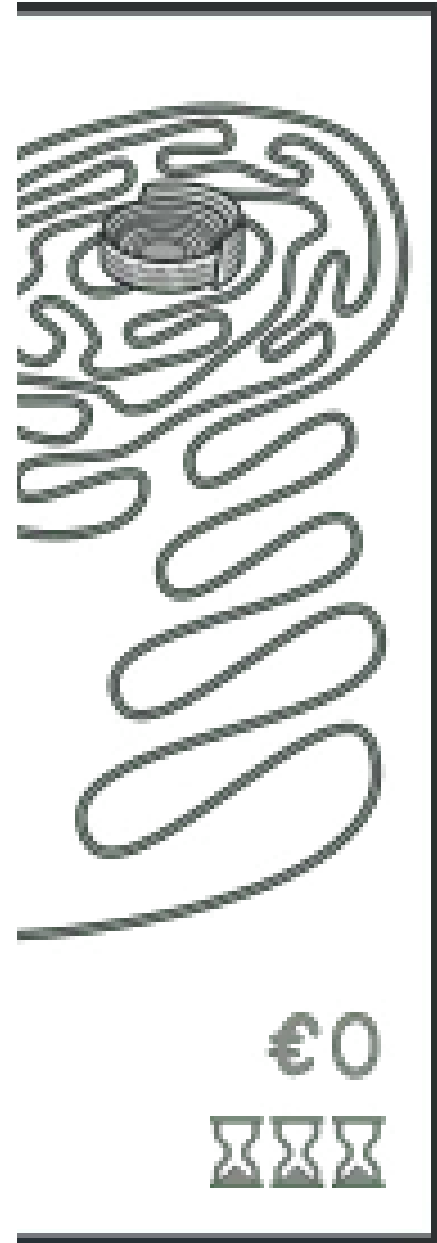
❏ **Clave:** Todos los iteradores deben implementar la **misma interfaz**. Esto hace que el código cliente sea compatible con cualquier tipo de colección o cualquier algoritmo de recorrido, siempre y cuando exista un iterador adecuado.

Si necesitas una forma particular de recorrer una colección, creas una **nueva clase iteradora** sin tener que cambiar la colección o el cliente.

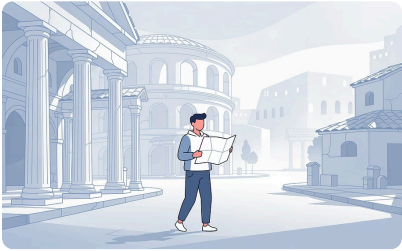
CAPÍTULO 4 · ANALOGÍA

Analogía en el Mundo Real

Planeas visitar **Roma** por unos días y ver todas sus atracciones. Pero una vez allí, podrías perder mucho tiempo dando vueltas, incapaz de encontrar siquiera el Coliseo.



Tres Formas de Pasear por Roma



Direcciones aleatorias

Caminar sin rumbo fijo por la ciudad, confiando en tu sentido de orientación.



Guía virtual (app)

Comprar una aplicación de guía para tu smartphone. Es buena, barata y puedes quedarte en sitios interesantes todo el tiempo que quieras.

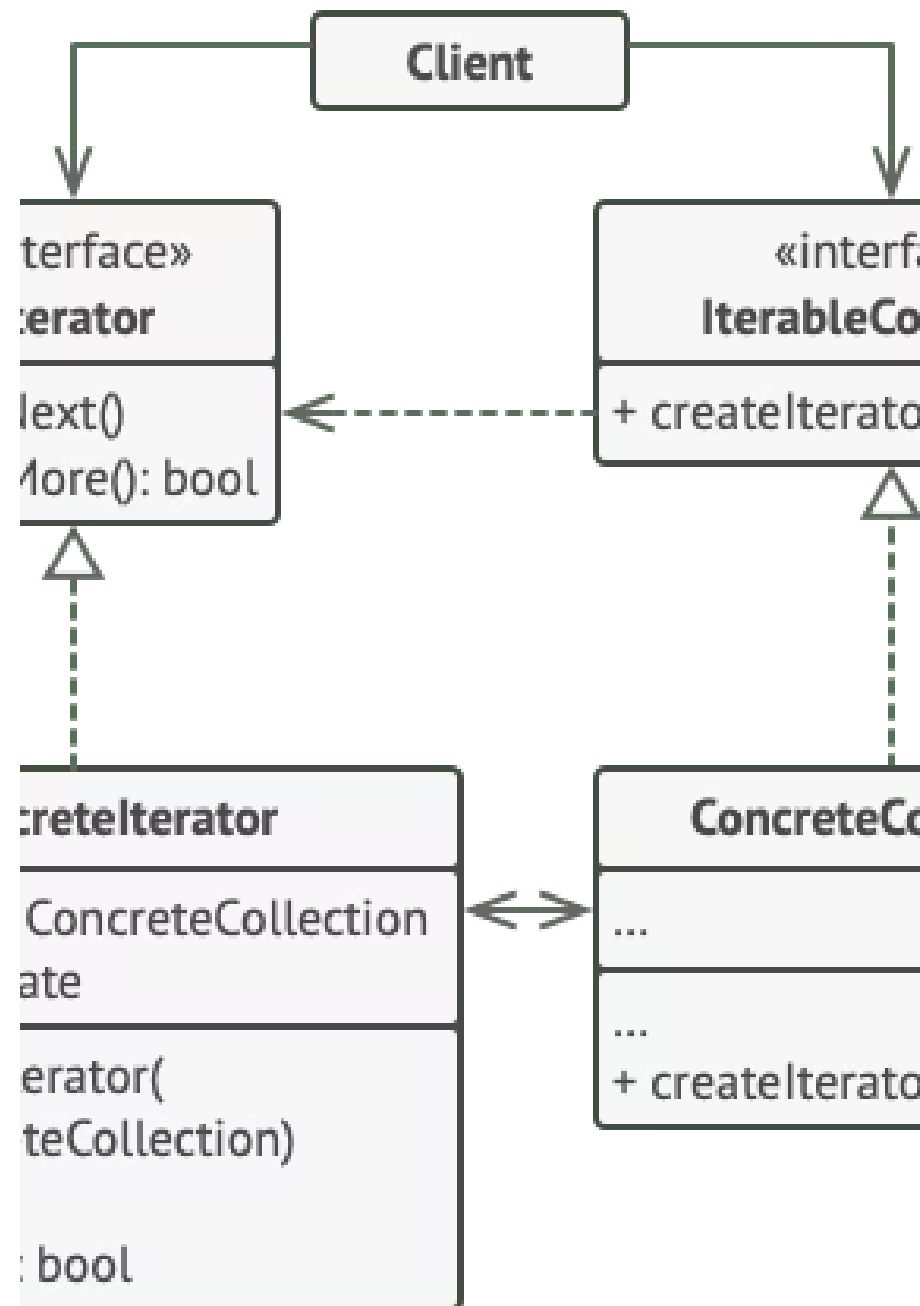


Guía local humano

Contratar un guía que conozca la ciudad como la palma de su mano. Adapta la visita a tus gustos, pero es más caro.

☐ Todas estas opciones actúan como **iteradores** sobre la amplia colección de visitas y atracciones de Roma.

Estructura del Patrón Iterator



Componentes de la Estructura

1 Interfaz Iteradora

Declara las operaciones necesarias para recorrer una colección: extraer el siguiente elemento, recuperar la posición actual, reiniciar la iteración, etc.

2 Iteradores Concretos

Implementan algoritmos específicos para recorrer una colección. El objeto iterador controla el progreso del recorrido por su cuenta, permitiendo recorridos independientes.

3 Interfaz Colección

Declara uno o varios métodos para obtener iteradores compatibles. El tipo de retorno debe declararse como la interfaz iteradora.

Componentes de la Estructura (cont.)

1 Colecciones Concretas

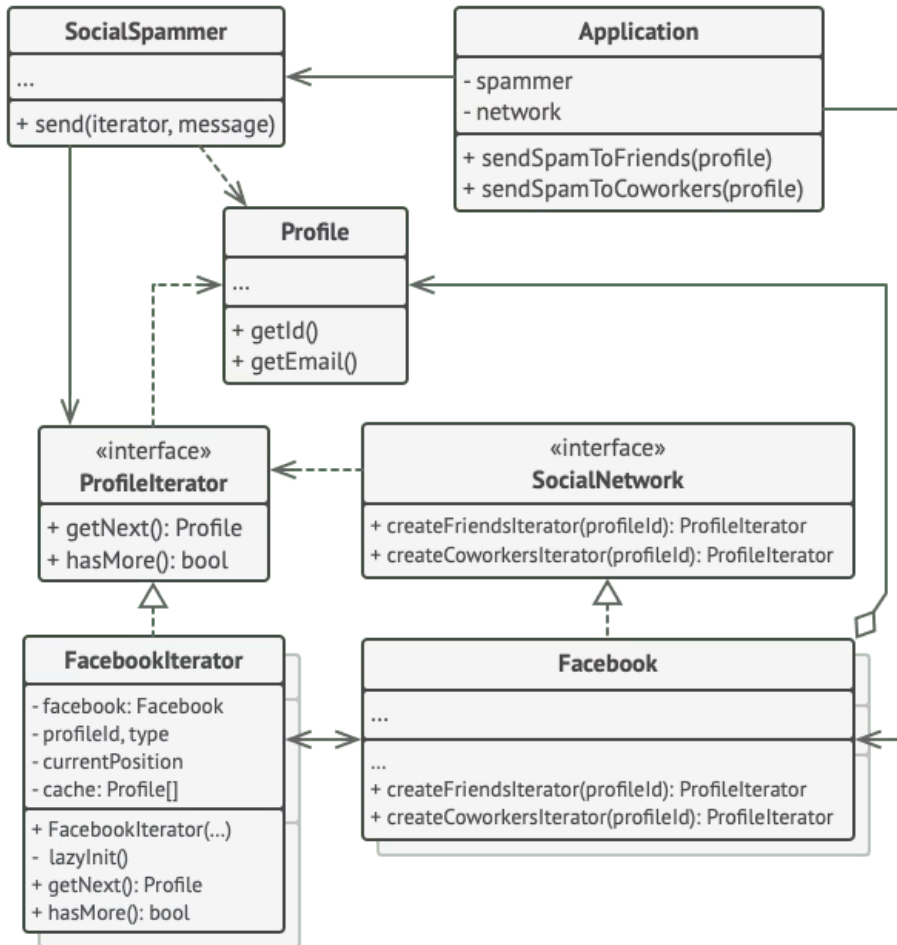
Devuelven nuevas instancias de una clase iteradora concreta particular cada vez que el cliente solicita una. El resto del código de la colección está en la misma clase.

2 El Cliente

Debe funcionar con colecciones e iteradores a través de sus **interfaces**. De este modo, no se acopla a clases concretas, permitiendo utilizar varias colecciones e iteradores con el mismo código.

- ❏ Normalmente, los clientes no crean iteradores por su cuenta; los obtienen de las colecciones. Sin embargo, en algunos casos, el cliente puede crear uno directamente, como cuando define su propio iterador especial.

Ejemplo: Iteración de Perfiles Sociales



En este ejemplo, el patrón Iterator se utiliza para recorrer un tipo especial de colección que encapsula el acceso al **grafo social de Facebook**.

La colección proporciona varios iteradores que recorren perfiles de distintas formas:

- El iterador '**amigos**' recorre los amigos de un perfil dado
- El iterador '**colegas**' omite amigos que no trabajen en la misma empresa

Ambos implementan una interfaz común que permite extraer perfiles sin profundizar en detalles como la autenticación y el envío de solicitudes REST.

Pseudocódigo: Interfaz SocialNetwork

La interfaz de colección declara un método fábrica para producir iteradores. Se pueden declarar varios métodos si hay distintos tipos de iteración disponibles.

```
interface SocialNetwork is
  method createFriendsIterator(profileId):ProfileIterator
  method createCoworkersIterator(profileId):ProfileIterator
```

Pseudocódigo: Clase Facebook

Cada colección concreta está acoplada a un grupo de clases iteradoras concretas que devuelve, pero el cliente no lo está, ya que la firma de estos métodos devuelve **interfaces iteradoras**.

```
class Facebook implements SocialNetwork is
// ... El grueso del código de la colección debe ir aquí ...

// Código de creación del iterador.
method createFriendsIterator(profileId) is
return new FacebookIterator(this, profileId, "friends")

method createCoworkersIterator(profileId) is
return new FacebookIterator(this, profileId, "coworkers")
```


Pseudocódigo: Interfaz Profilelterator

La interfaz común a todos los iteradores define los métodos esenciales para el recorrido.

```
interface Profilelterator is  
  method getNext():Profile  
  method hasMore():bool
```

Pseudocódigo: Clase FacebookIterator

La clase iteradora concreta. El iterador necesita una referencia a la colección que recorre y almacena su propio **estado de iteración**.

class FacebookIterator implements ProfileIterator is

private field facebook: Facebook

private field profileId, type: string

private field currentPosition

private field cache: array of Profile

constructor FacebookIterator(facebook, profileId, type) is

this.facebook = facebook

this.profileId = profileId

this.type = type

private method lazyInit() is

if (cache == null)

cache = facebook.socialGraphRequest(profileId, type)

method getNext() is

if (hasMore())

result = cache[currentPosition]

currentPosition++

return result

method hasMore() is

lazyInit()

return currentPosition < cache.length

Pseudocódigo: SocialSpammer y Application

Puedes pasar un iterador a una clase cliente en lugar de darle acceso a una colección completa. Así no expones la colección y puedes **cambiar la forma de recorrido en tiempo de ejecución**.

```
class SocialSpammer is
  method send(iterator: ProfileIterator, message: string) is
    while (iterator.hasMore())
      profile = iterator.getNext()
      System.sendEmail(profile.getEmail(), message)
```

```
class Application is
  field network: SocialNetwork
  field spammer: SocialSpammer
```

```
  method config() is
    if working with Facebook
      this.network = new Facebook()
    if working with LinkedIn
      this.network = new LinkedIn()
    this.spammer = new SocialSpammer()
```

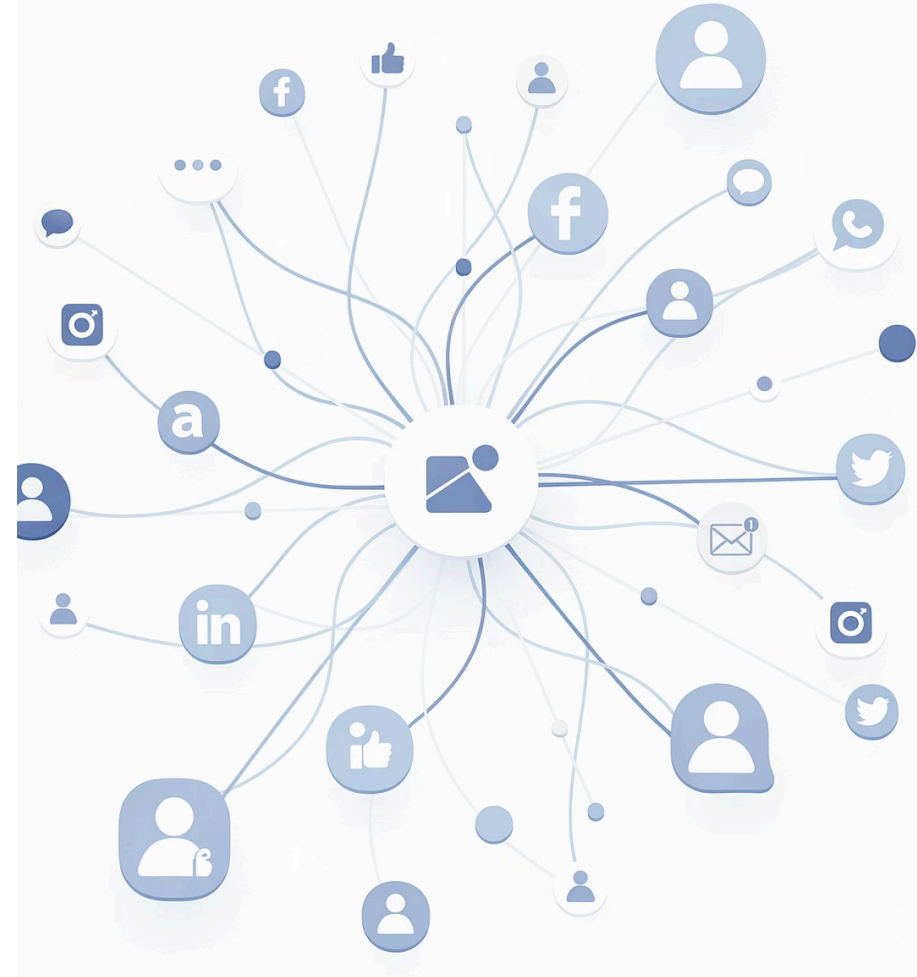
```
  method sendSpamToFriends(profile) is
    iterator = network.createFriendsIterator(profile.getId())
    spammer.send(iterator, "Very important message")
```

```
  method sendSpamToCoworkers(profile) is
    iterator = network.createCoworkersIterator(profile.getId())
    spammer.send(iterator, "Very important message")
```

Ventaja Clave del Pseudocódigo

El código cliente **no está acoplado a clases concretas** porque sólo trabaja con colecciones e iteradores a través de interfaces.

- ❏ Si decides conectar tu aplicación a una **nueva red social**, sólo necesitas proporcionar nuevas clases de colección e iteradoras, **sin cambiar el código existente**.



¿Cuándo Utilizar el Patrón Iterator?



Ocultar complejidad

Cuando tu colección tenga una estructura compleja a nivel interno pero quieras ocultar su complejidad a los clientes, ya sea por conveniencia o por seguridad.



Reducir duplicación

El código de algoritmos de iteración no triviales tiende a ser voluminoso. Mover el código de recorrido a iteradores designados hace el código más breve y limpio.



Recorrer distintas estructuras

Cuando quieras que tu código pueda recorrer distintas estructuras de datos, o cuando los tipos de estas estructuras no se conozcan de antemano.

Cómo Implementar el Patrón

01

Declarar la interfaz iteradora

Como mínimo, debe tener un método para extraer el siguiente elemento. Puedes añadir métodos para extraer el elemento previo, localizar la posición actual o comprobar el final de la iteración.

02

Declarar la interfaz de colección

Describe un método para buscar iteradores. El tipo de retorno debe ser igual al de la interfaz iteradora. Puedes declarar métodos similares si planeas tener varios grupos distintos de iteradores.

03

Implementar iteradores concretos

Un objeto iterador debe estar vinculado a una única instancia de la colección. Normalmente, este vínculo se establece a través del constructor del iterador.

04

Implementar la interfaz de colección

Proporcionar al cliente un atajo para crear iteradores personalizados. El objeto de colección debe pasarse a sí mismo al constructor del iterador.

05

Repasar el código cliente

Sustituir todo el código de recorrido de la colección por el uso de iteradores. El cliente busca un nuevo objeto iterador cada vez que necesita recorrer los elementos.

Ventajas del Patrón Iterator



Responsabilidad única

Puedes limpiar el código cliente y las colecciones extrayendo algoritmos de recorrido voluminosos y colocándolos en clases independientes.



Abierto/Cerrado

Puedes implementar nuevos tipos de colecciones e iteradores y pasarlos al código existente sin descomponer nada.



Recorrido en paralelo

Puedes recorrer la misma colección en paralelo porque cada objeto iterador contiene su propio estado de iteración.



Iteración diferida

Puedes retrasar una iteración y continuar cuando sea necesario, gracias al estado independiente de cada iterador.

Desventajas del Patrón Iterator



Excesivo para colecciones sencillas

Aplicar el patrón puede resultar excesivo si tu aplicación funciona únicamente con colecciones sencillas.



Menor eficiencia

Utilizar un iterador puede ser menos eficiente que recorrer directamente los elementos de algunas colecciones especializadas.

Relaciones con Otros Patrones



Iterator + Composite

Puedes utilizar Iteradores para recorrer árboles Composite.



Iterator + Factory Method

Permite que las subclases de la colección devuelvan distintos tipos de iteradores compatibles con las colecciones.



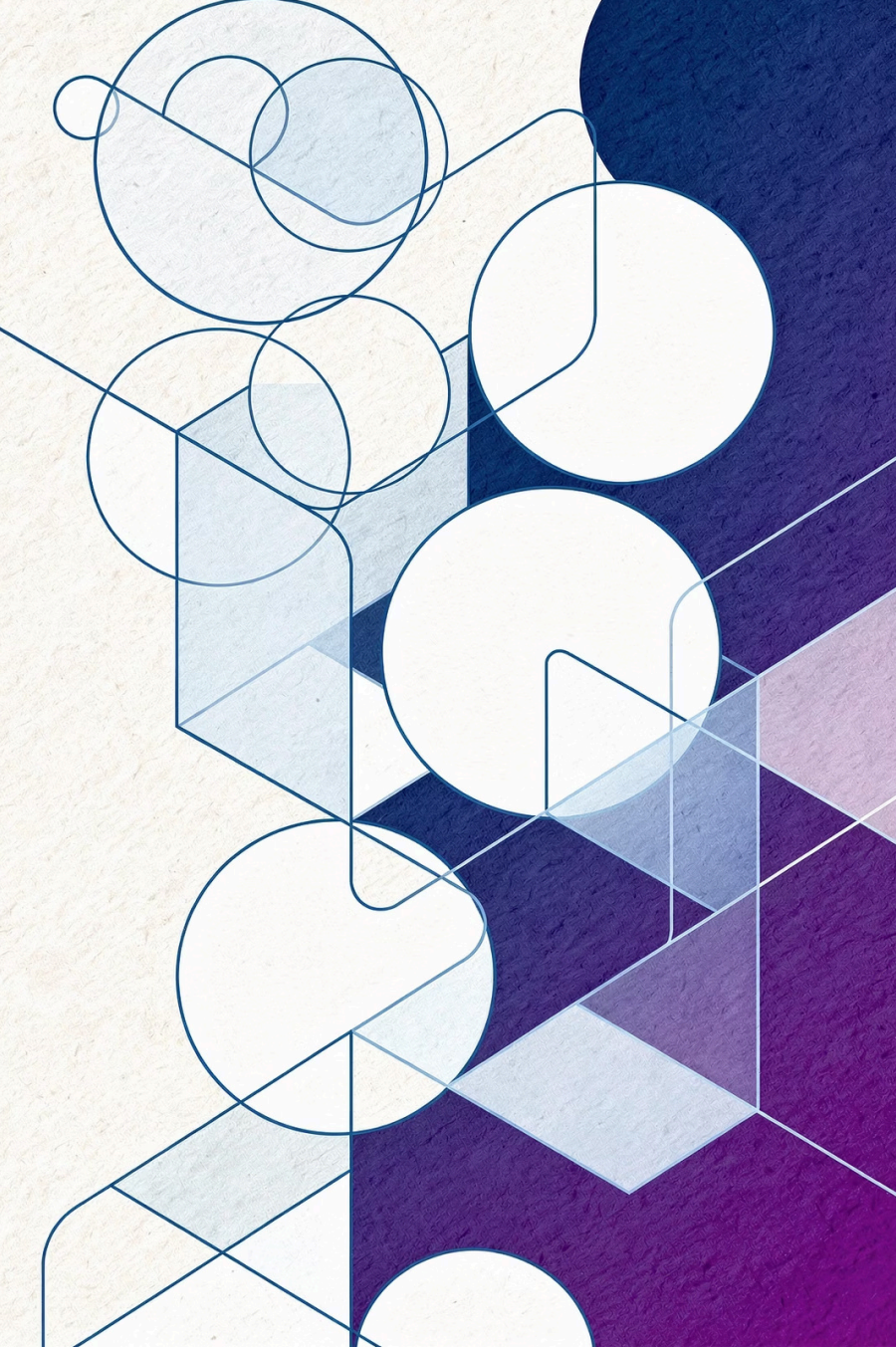
Iterator + Memento

Puedes usar Memento junto con Iterator para capturar el estado de la iteración actual y reanudarla si fuera necesario.



Iterator + Visitor

Puedes utilizar Visitor junto con Iterator para recorrer una estructura de datos compleja y ejecutar operaciones sobre sus elementos, incluso con clases distintas.



Resumen: Patrón Iterator

Propósito

Recorrer colecciones
sin exponer su
estructura interna

Clave

Interfaz común para
todos los iteradores

Beneficio

Desacoplamiento
entre cliente y
colección

El patrón Iterator separa la lógica de recorrido de la colección, promoviendo el **principio de responsabilidad única** y el **principio abierto/cerrado**.